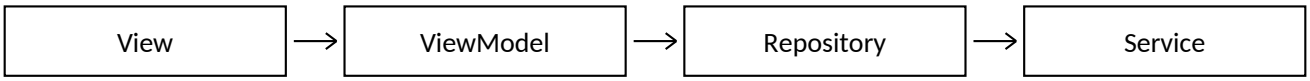
Bende Var: Architecture and Protocols

Team 2 · MTM4186 Mobile Programming · Milestone 1

Bende Var is an Android app built with Flutter, with Firebase as the backend. People in a closed community (a school, a dormitory or a workplace) post **requests** for things they want to borrow or buy and **offers** for things they can lend or sell. Others answer with "Bende Var" or "İhtiyacım Var", agree in the chat, confirm the handover and the return with a QR code, and rate each other. Payment is made outside the app, face to face or by IBAN.

1. Software design pattern: MVVM

We plan to use MVVM (Model, View, ViewModel), the architecture recommended in the official Flutter app architecture guide. The app has a UI layer (Views and ViewModels) and a data layer (Repositories and Services). Each part only talks to the part next to it, so screens never call Firebase directly.



Calls go from left to right. Data comes back from right to left as a Future or a Stream.

Part	How it applies to Bende Var
View	The screens designed in Figma. They show the current state (data, loading, empty, error) and send user actions to their ViewModel. No business logic.
ViewModel	A ChangeNotifier class (provider package) that keeps the state of one screen or flow, such as the step of a form or the selected tab, and exposes commands such as publishListing, sendMessage or acceptOffer.
Repository	The single source of truth for one type of data (listings, responses, messages, transactions). Turns raw data into models and handles errors.
Service	A thin wrapper around Firebase or a device API (authentication, database, images, notifications). Holds no state.
Model	Plain Dart data classes: User, Community, Listing, Response, Message, Transaction, Review.

2. Modules

Each team member designed one module in Figma. Every module follows the same structure.

Module (Figma screens)	Main Views	Main ViewModels and Repositories
Account and settings (1.xx)	Login, register, email verification, password reset, settings, communities	AuthViewModel, SettingsViewModel; AuthRepository, UserRepository, CommunityRepository
Home and favorites (2.xx)	Home with offers and requests, search, filters, listing detail, favorites	HomeViewModel, FavoritesViewModel; ListingRepository, FavoriteRepository
Creating listings (3.xx)	Offer and request forms, photos, preview, owner view of a listing	CreateListingViewModel, MyListingViewModel; ListingRepository, ImageRepository
Messages and notifications (4.xx)	Messages list, chat, notifications	ChatViewModel, NotificationsViewModel; ChatRepository, NotificationRepository
Profile and transactions (5.xx)	Profile, my loans, incoming offers, QR handover and return, ratings	ProfileViewModel, TransactionViewModel; ResponseRepository, TransactionRepository

Example. A user taps "Bende Var" on a request. The ViewModel calls ResponseRepository to save the response and ChatRepository to open a chat. The owner is notified, and when they accept, TransactionRepository creates the transaction. The handover and the return are confirmed by scanning a QR code.

3. Key protocols and interfaces

Protocol / interface	Purpose
Firebase Authentication	Register and log in with the institution email, email verification and password reset. Each user gets a unique id (uid).
Cloud Firestore (real time listeners)	Main database. Listeners send every change, so lists, chats and statuses update without refreshing. Offline persistence is on by default on Android.
Firestore Security Rules	Server side access control: only community members see its listings, only the owner edits a listing, only the two users read their chat.
Firebase Cloud Messaging (FCM)	Push notifications for new responses and messages. Sending needs a server side function, so it depends on the Firebase plan (see section 4).
flutter_local_notifications	Return reminders scheduled on the phone before the due date.
image_picker and image upload (HTTPS)	Taking or choosing listing photos. Photos are uploaded and only their URLs are saved in Firestore.
qr_flutter and mobile_scanner	One phone shows a QR code, the other scans it to confirm the handover or the return (see section 4).
Repository interfaces (Dart abstract classes)	Contracts between modules. Modules depend on these interfaces, not on Firebase, so they can be built and tested separately.

```
abstract class ListingRepository {
  Stream<List<Listing>> watchListings(String communityId);
  Future<String> publishListing(Listing listing);
}
abstract class ChatRepository {
  Stream<List<Message>> watchMessages(String conversationId);
  Future<void> sendMessage(String conversationId, String text);
}
abstract class TransactionRepository {
  Future<String> createTransaction(Response accepted);
  Future<void> confirmHandover(String transactionId, String code);
}
```

4. Scope and decisions that may change

For now we plan to use Firebase's free Spark plan; we may move to the paid Blaze plan later if the project needs it. The first version focuses on the core flow: accounts, communities, listings, responses, chat, in-app notifications and transactions. The parts below depend on the Firebase plan or on time, so they may change during development:

- **Push notifications (FCM):** sending needs Cloud Functions, which needs the paid Blaze plan. If we stay on the free plan, notifications are shown only inside the app.
- **Listing photos:** Cloud Storage also needs the Blaze plan. The alternative is a free image host (for example Cloudinary). If this is not possible, listings start without photos.
- **QR handover:** if camera scanning causes problems, the handover and the return are confirmed with the 6 digit code or a confirmation button on both phones.

Because Views and ViewModels only use repository interfaces, such a change only affects one Repository or Service. The screens and the MVVM structure stay the same.

References. Flutter: Guide to app architecture (docs.flutter.dev/app-architecture/guide). Firebase: Get realtime updates with Cloud Firestore (firebase.google.com/docs/firestore/query-data/listen), Access data offline (firebase.google.com/docs/firestore/manage-data/enable-offline), Firebase Cloud Messaging for Flutter (firebase.google.com/docs/cloud-messaging/flutter/client). Packages on pub.dev: provider, image_picker, flutter_local_notifications, qr_flutter, mobile_scanner.